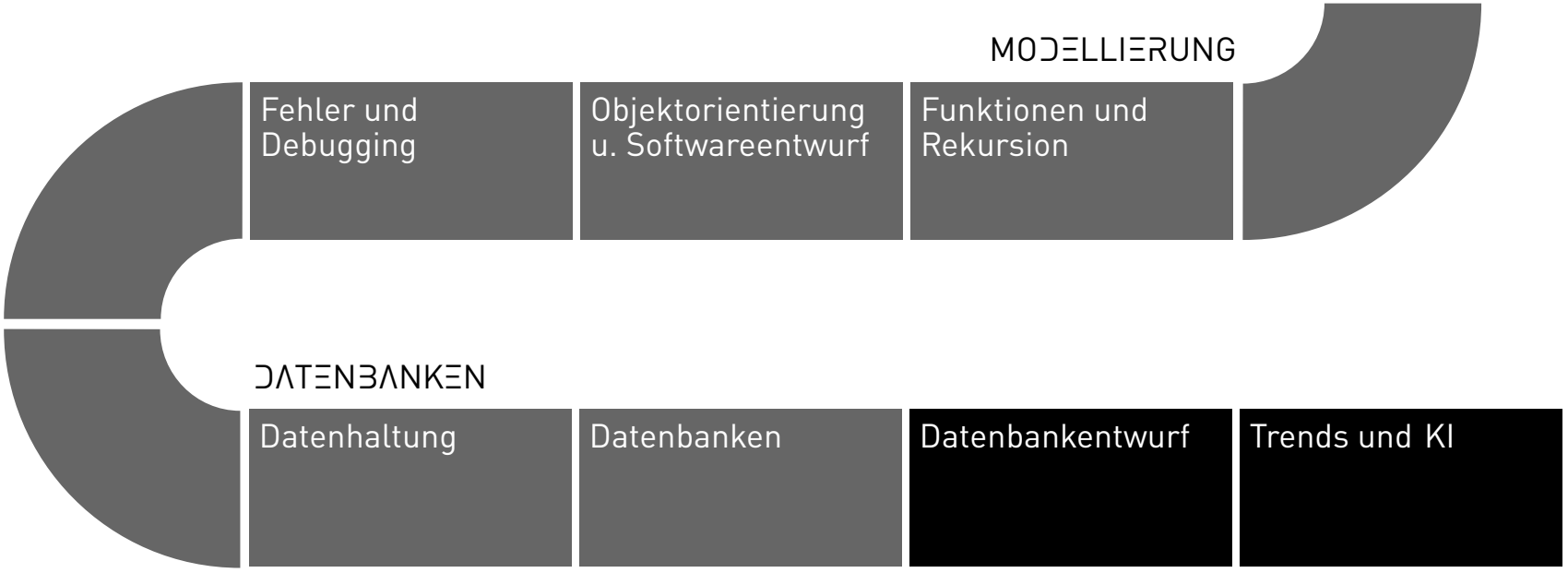
Programming and databases

Joern Ploennigs

SQL Select

MIJOURNEY: DEEP DIVE INTO A BUSY SCENE

Procedure


MODELLIERUNG
Fehler und Debugging
Objektorientierung u. Softwareentwurf
Funktionen und Rekursion
DATENBANKEN
Datenhaltung
Datenbanken
Datenbankentwurf
Trends und KI

How do you obtain data from the tables?

- The tables are not read sequentially like files, but we perform *query operations*.
- These compute a new *result table* from a set of tables.
- These operations can be combined in a way similar to familiar arithmetic operations.
- So we use a system of computations over tables: a *relational algebra*

How do we put these into practice?

- Historically, dedicated *query languages* have developed here, rather than just add-on functions and libraries in other programming languages
- The most popular of these languages is the *Structured Query Language (SQL)*
- It has a syntax specifically designed for the operations allowed here
- In practice, it is considerably more focused than Python

SQL (Structured Query Language)

- *1975* SEQUEL = Structured English Query Language
- *since 1986* SQL ANSI standard, since 1987 ISO standard
- *1992* SQL 2 (SQL-92) new ISO standard
- *1999* SQL 3 (SQL:1999) new ISO standard
- *2016* SQL 7 (SQL:2011) new ISO standard
- *2019* SQL 9 (SQL:2019) stable release
- *SQL:2016* ISO/IEC 9075-1:2016 is the current ISO standard

Projection

- Given the relation Municipalities {MunicipalityID, Name, Population}
- Projection means selecting certain columns by explicit listing
- Example: MunicipalityID, Name

MunicipalityID	Name	Population
1	Dummerstorf	7,329
2	Graal-Müritz	4,278
3	Sanitz	5,831

Projection in SQL

Syntax:

```
SELECT ... FROM ...
```

- *SELECT* – Selects the required columns (using * selects all columns)
- *FROM* – Specifies the relations from which to select

```
SELECT GemeindeID, Name
FROM Gemeinden
```

Result:

MunicipalityID	Name
1	Dummerstorf
2	Graal-Müritz
3	Sanitz

Selection

- *Selection* means selecting specific rows based on a condition
- For the conditions, as usual, comparison operators are used
- Condition: Population > 5000

Municipality ID	Name	Population
1	Dummerstorf	7,329
~~2~~	~~Graal-Müritz~~	~~4,278~~
3	Sanitz	5,831

Selection in SQL

Syntax:

```
SELECT ... FROM ... WHERE ...
```

- *WHERE* – filters the rows in the selected tables according to specified conditions

Example:

```
SELECT * FROM Gemeinden
WHERE Einwohner > 5000
```

Municipality ID	Name	Population
1	Dummerstorf	7.329
3	Sanitz	5.831

Natural Join

With a *JOIN*, we combine rows from both tables wherever foreign keys and primary keys match.

Municipalities Table

MunicipalityID	Name	Population
1	Dummerstorf	7,329
2	Graal-Müritz	4,278
3	Sanitz	5,831

Primary key: MunicipalityID

Buildings Table

BuildingID	Building Type	MunicipalityID
5000	Gas Station	2
5001	Hotel	1
5002	Church	2

Foreign Key: MunicipalityID

Natural Join - Result

Joining tables by matching columns:

Municipalities table

MunicipalityID	Name	Population
1	Dummerstorf	7.329
2	Graal-Müritz	4.278
3	Sanitz	5.831

Primary key: MunicipalityID

Buildings table

BuildingID	Building Type	MunicipalityID
5000	Gas Station	2
5001	Hotel	1
5002	Church	2

Foreign key: MunicipalityID

Result

BuildingID	Building Type	MunicipalityID	Name	Population
5000	Gas Station	2	Graal-Müritz	4.278
5001	Hotel	1	Dummerstorf	7.329
5002	Church	2	Graal-Müritz	4.278

Important: Tuples whose attribute value in the respective column of the other relation does not appear will also not appear in the result table.
In the example: Sanitz does not appear in the result because none of the existing buildings are built here.

Natural Join - in SQL

Syntax:

```
SELECT ... FROM ... NATURAL JOIN
```

- *NATURAL JOIN* – Connects two tables via the natural join

Example:

```
SELECT * FROM Bauwerke
NATURAL JOIN Gemeinden
```

Other join forms in SQL

Kreuzprodukt:

```
SELECT … FROM Table1, Table2
```

- If you separate with a comma instead of NATURAL JOIN, a *Cross product* is formed – all rows are joined with all rows!

Sinnvoll um mehrere Tabellen nach bestimmten Bedingungen zu kombinieren:

```
SELECT … FROM Table1, Table2
WHERE Table1.Foreign = Table2.Primary
```

Can this also produce the NATURAL JOIN?

Renaming

- Sometimes the same attribute values in different tables have different meanings, especially when forming joins.
- Given the relation Buildings {BuildingID, BuildingType, Name}
- Join: Buildings × Municipalities
- Columns: BuildingID, BuildingType, Name
- *Renaming:* Name ⟶ MunicipalityName

BuildingID	BuildingType	MunicipalityName
5000	Gas Station	Graal-Müritz
5001	Hotel	Dummerstorf
5002	Church	Graal-Müritz

Aliasing in SQL

With the `AS` keyword:

```
SELECT tbl1.attr1 AS IdentificationNumber
FROM Table1 AS tbl1
```

The example from earlier:

```
SELECT StructureID, StructureType, Name AS MunicipalityName
FROM Structures
NATURAL JOIN Municipalities
```

This is possible in both the `SELECT` and the `FROM` clauses by using the `AS` keyword.

Aggregate Functions

- Aggregate column values according to a defined procedure
- They typically form a 1×1 table as the result
- If more than one value should appear, it must be aggregated according to a defined condition
- If there is a condition, the rows represent different cases
- *Common aggregate functions:* sums or counts

MunicipalityID	Name	Population
1	Dummerstorf	7,329
2	Graal-Müritz	4,278
3	Sanitz	5,831

Sum: 17,438

Aggregate Functions in SQL

COUNT function counts the number of rows:

```
SELECT COUNT(GemeindeID) FROM
Gemeinden
```

```
*COUNT: 3*
```

SUM function sums a column numerically:

```
SELECT SUM(Einwohner) FROM Gemeinden
```

```
*SUM: 17,438*
```

Conditional Aggregation - in SQL

- When you aggregate by a particular attribute, only those rows that share the same attribute value are aggregated (summed, counted, averaged, etc.).
- For this, the GROUP BY clause in SQL is used.

Example: A table named Mitarbeiter contains honorarium payments to employees; for each payment there is a row. One now wants to output the average honorarium for each employee.
Gegeben die Relation Mitarbeiter{MitarbeiterID, Name, Honorar}

```
SELECT MitarbeiterID, Name, AVG(Honorar)
FROM Mitarbeiter
GROUP BY MitarbeiterID
```

Further operations: Sorting & Count

Sorting:

```
SELECT * FROM Gemeinden
ORDER BY Einwohner DESC
```

- `ORDER BY` with `ASC` (ascending) or `DESC` (descending)

Limiting the result set:

```
SELECT * FROM Gemeinden
ORDER BY Einwohner DESC
LIMIT 2
```

- `LIMIT` truncates the result set to a certain number of elements
- Here: The two municipalities with the highest population

Combining Conditions

Conditions can be combined, just like in Python, with `AND` and `OR`:

```
SELECT * FROM Bauwerke
NATURAL JOIN Gemeinden
WHERE Einwohner > 5000 AND
Bauwerkstyp = "Hotel"
```

Result:

BuildingID	BuildingType	MunicipalityID	Name	Popu
5001	Hotel	1	Dummerstorf	7,32

Outlook: SQL - More than a Query Language

So far: Only a single aspect of SQL (`SELECT` ...)
SQL offers far more functionalities:

- *DML (Data Manipulation Language):*
  - Querying and modifying data in tables (`SELECT, UPDATE, DELETE` ...)
- *DDL (Data Definition Language):*
  - Defining and managing schemas and tables (`CREATE, ALTER, DROP` ...)
- *DCL (Data Control Language):*
  - User privileges (`GRANT, REVOKE` ...)
  - Transaction control (`COMMIT, ROLLBACK` ...)

Questions?

PROGRAMMIERUNG UND DATENBANKEN